# PROGRAMMING METHODS - LABORATORY 3

## Program 01

Write a memory- and computation-efficient program operating on an integer array T[n] (where 2 < n < 2^15) sorted in non-decreasing order. The program should perform the following operations:

- calculate the number of array elements equal to a given value with worst-case complexity O(log n);
- calculate the index of an array element equal to a given value using interpolation search;
- remove duplicates with complexity O(n).

Divide the program into .cpp and .h files containing thematically organized functions.

## Input

Data are stored in a text file containing consecutive lines. The first line contains an integer from 1 to 2^15, denoting the number of data sets. The data sets follow.

Each data set contains:

- a positive integer from 1 to 2^15 denoting the number of values in the data set;
- the main data of the set, in the number read previously, consisting of integers from -2^48 to +2^48, given in non-decreasing order;
- a positive integer from 1 to 2^15 denoting the number of queries for the searched value;
- integers from -2^48 to +2^48 denoting the queried values, in the number read previously.

## Output

For each data set and for each queried value, write the following lines to a text file:

- First line: pairs of numbers in the form (i j), separated by spaces. i is the number of array elements equal to the given value. j equals -1 if the array does not contain the given value; otherwise j is the index of an element equal to the given value, found using interpolation search. If all array elements are equal to the given value, i is 0.
- Next lines: print at most the first 200 elements of the array after duplicate removal, separated by spaces, 50 elements per line.
- The output for every data set starts on a new line.

## Examples

### in1.txt / out1.txt

```
Input:
1
12
-1 1 2 2 2 3 5 5 7 7 9 9
10
1 2 3 -1 4 9 5 6 7 8

Output:
(1 1) (3 3) (1 5) (1 0) (0 -1) (2 10) (2 6) (0 -1) (2 8) (0 -1)
-1 1 2 3 5 7 9
```

### in2.txt / out2.txt

```
Input:
1
10
1 2 2 2 2 2 3 3 3 3
5
1 2 3 4 0

Output:
(1 0) (5 4) (4 9) (0 -1) (0 -1)
1 2 3
```

### in3.txt / out3.txt

```
Input:
1
10
0 0 0 0 0 0 0 0 0 1
4
0 -1 1 2

Output:
(9 0) (0 -1) (1 9) (0 -1)
0 1
```

### in4.txt / out4.txt

```
Input:
1
10
1 1 1 1 1 1 1 1 1 1
3
1 0 2

Output:
(10 0) (0 -1) (0 -1)
1
```

Note: generate your own text file containing about 1000 elements and search for about 100 elements.